

WHATNEXT VISION MOTORS

Salesforce CRM Implementation Project Report

Project Title: WhatNext Vision Motors – Salesforce CRM System

Project Type: Team Project

Developed By:

- Praveen Alexander
- Lokeshwar C
- Mira Asmitha S
- Venkateswaran A
- Muthukumaran R

Course: Naan Mudhalvan – Salesforce

Institution: Tagore Engineering College

Guide: Asst. Professor Suganthi J

Date: 03-04-2026

ABSTRACT

WhatsNext Vision Motors is undertaking a transformative Salesforce CRM implementation aimed at enhancing customer experience and operational efficiency within the automotive sector. Central to the solution is a data model integrating vehicle inventory, dealer locations, and customer management. The CRM leverages automated workflows and Apex triggers to enforce business logic—blocking orders for out-of-stock vehicles and auto-assigning new orders to the nearest dealer based on customer geolocation.

Complementing these real-time safeguards, the project integrates scheduled processes: batch Apex updates stock availability across bulk orders and scheduled Apex dispatches email reminders for test drives and stock replenishment. Together, these components form a cohesive system that offers transparent, accurate order tracking—marking statuses as “Pending” or “Confirmed” depending on inventory levels.

The technical implementation includes structured data modeling of vehicle and dealer records, Lightning App Builder for streamlined UI interactions, record-triggered flows for dynamic logic execution, and robust Apex code for validation and automation.

By reducing manual intervention, minimizing customer friction, and improving responsiveness to stock changes, the project is set to significantly elevate customer satisfaction and internal productivity propelling WhatsNext Vision Motors toward more agile and customer-centric operations.

OBJECTIVE

The objective of this project is to enhance the customer ordering experience by leveraging Salesforce automation and intelligent data handling. It aims to streamline dealer assignment, ensure real-time stock validation, and provide accurate order status updates. By doing so, WhatsNext Vision Motors seeks to improve customer satisfaction and operational efficiency across its mobility services.

- **Enhanced Customer Experience:** Simplifying the ordering process with automated dealer suggestions and real-time updates.
- **Real-Time Stock Management:** Preventing out-of-stock orders through accurate inventory validation.
- **Automated Dealer Assignment:** Assigning orders to the nearest dealer based on customer location.
- **Efficient Order Processing:** Using scheduled processes to update order statuses and streamline fulfillment.
- **Smart Salesforce Integration:** Implementing Apex, Flows, and Batch Jobs to automate and scale operations.

TECHNOLOGY DESCRIPTION

Salesforce:-

Salesforce is a cloud-based Customer Relationship Management (CRM) platform that enables businesses to manage customer interactions, data, and processes efficiently. In this project, it is used to store and manage vehicle details, dealer locations, and customer orders. The platform supports automation through Flows, Apex triggers, and batch jobs to ensure accurate stock validation and smart dealer assignment. By leveraging Salesforce, WhatsNext Vision Motors enhances operational efficiency and delivers a seamless, customer-centric ordering experience.

Custom Objects:-

Objects in salesforce are like tables in a database. Custom Objects are created to store specific data.

Example:

- Vehicle_c - Stores vehicle details
- Vehicle_Dealer_c - Stores authorized dealer info
- Vehicle_Customer_c - Stores customer details
- Vehicle_Order_c - Tracks vehicle purchases
- Vehicle_Test_Drive_c - Tracks test drive bookings
- Vehicle_Service_Request_c - Tracks vehicle servicing requests

Tabs:-

Tabs are used to display data in the Salesforce UI

Example: A tab for vehicle_c allows users to easily view, create, and manage vehicle

Custom APP:-

An App in Salesforce is a collection of tabs grouped together for a specific business purpose, allowing users to access related data and functionality in one place.

Fields:-

A **fields** in Salesforce is a specific piece of data stored in an object, much like a column in a spreadsheet. Each field holds a particular type of information such as text, date, number, or picklist values.

Example: Vehicle_Model_c – A **Picklist** field in the Vehicle_c object with options like *Sedan*, *SUV*, *EV*, etc.

Flows:-

Flows automate business logic without writing code. They can create or update records, and send notifications automatically.

Example:-

- A flow triggers an email alert whenever a new order is created.

Apex:-

Apex is Salesforce's Object-Oriented programming language. It allows developers to write custom logic .

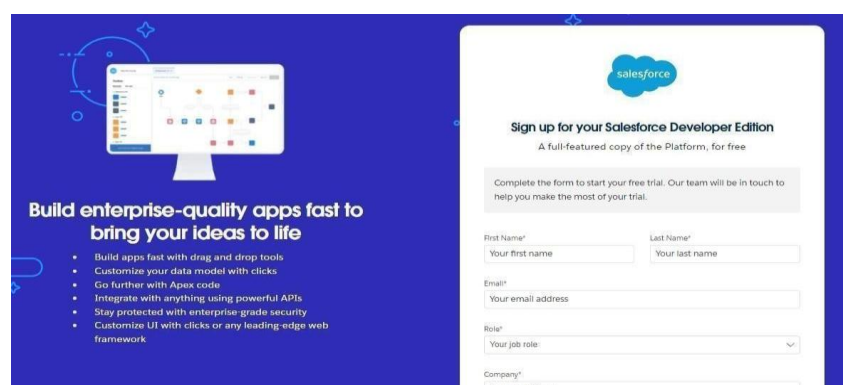
Example Triggers:

- A trigger automatically validates stock availability when a new vehicle order is created.
- It prevents orders if the vehicle is out of stock.
- Once the order is confirmed, the trigger updates the stock quantity by reducing it.

DETAILED EXECUTION OF PROJECT PHASES

1. Developer Org Setup

A Salesforce Developer Org was Created Using <https://developer.salesforce.com/signup>



- The account was verified, password set , and access was granted to the Salesforce Setup page.

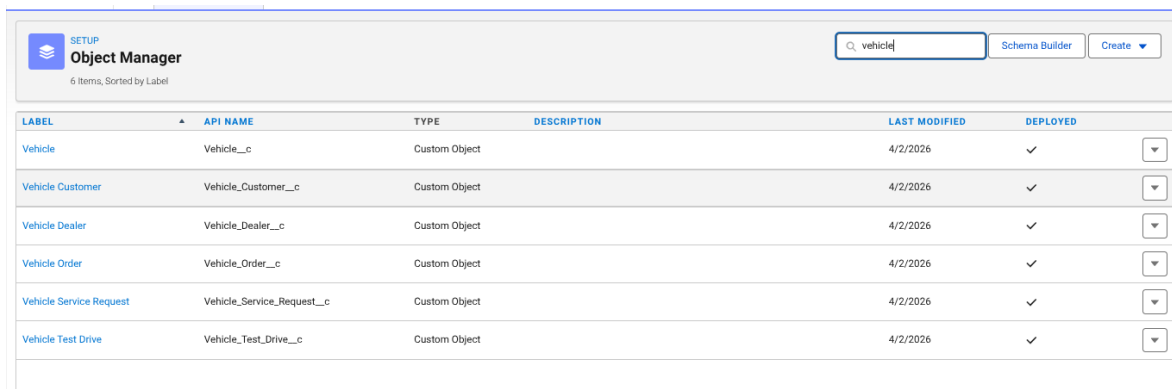
2. Custom Object Creation

Six Custom Objects were created to store business-critical data

- **Vehicle** – Stores vehicle details like model, stock, price, and status.
- **Vehicle Dealer** – Contains information about dealers such as location and contact details.
- **Vehicle Order** – Tracks customer orders, order dates, and order status.
- **Vehicle Customer** – Maintains customer details including contact and preferred vehicle type.
- **Vehicle Test Drive** – Records test drive schedules and status.
- **Vehicle Service Request** – Logs service requests, dates, issues, and progress status.

Steps followed:

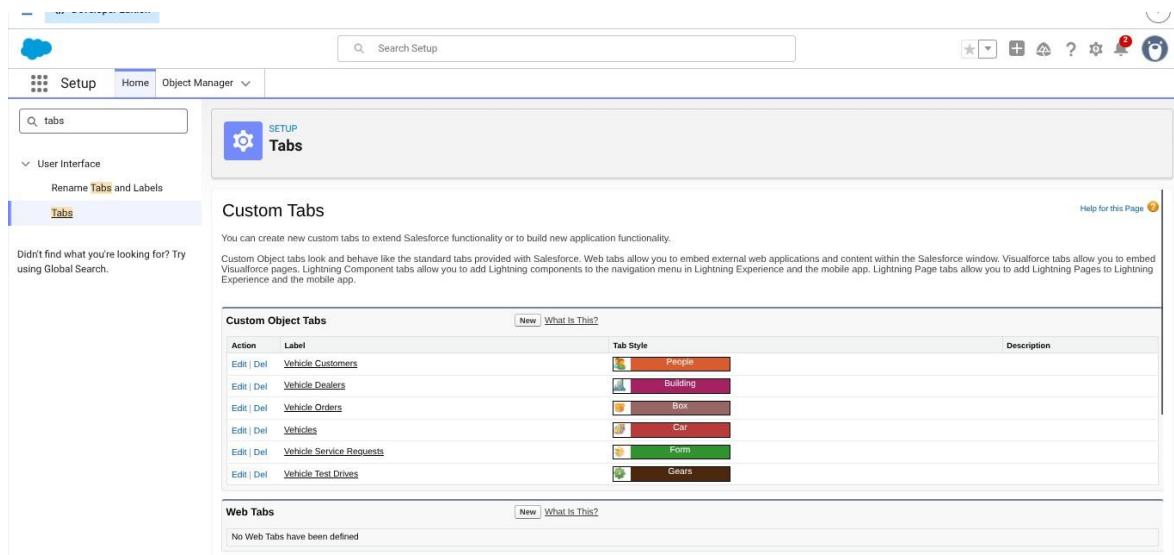
- Navigated to Setup → Object Manager → Create → Custom Object.
- Provided Label, Name, and enabled options for Reports and Search.
- Saved and created Tabs for each object.



The screenshot shows the Salesforce Object Manager interface. At the top, there's a search bar with 'vehicle' entered and buttons for 'Schema Builder' and 'Create'. Below the header, a table lists six custom objects. Each row includes a label, API name, type, description, last modified date, deployment status, and a dropdown menu.

LABEL	API NAME	TYPE	DESCRIPTION	LAST MODIFIED	DEPLOYED	
Vehicle	Vehicle__c	Custom Object		4/2/2026	✓	▼
Vehicle Customer	Vehicle_Customer__c	Custom Object		4/2/2026	✓	▼
Vehicle Dealer	Vehicle_Dealer__c	Custom Object		4/2/2026	✓	▼
Vehicle Order	Vehicle_Order__c	Custom Object		4/2/2026	✓	▼
Vehicle Service Request	Vehicle_Service_Request__c	Custom Object		4/2/2026	✓	▼
Vehicle Test Drive	Vehicle_Test_Drive__c	Custom Object		4/2/2026	✓	▼

3. Creating a Custom Tab



The screenshot shows the Salesforce Setup page for 'Custom Tabs'. The left sidebar has a search bar with 'tabs' and a list of tabs under 'User Interface'. The main content area has a header 'SETUP Tabs' and a description of custom tabs. Below this, there are two sections: 'Custom Object Tabs' and 'Web Tabs'. The 'Custom Object Tabs' section has a table with columns for Action, Label, Tab Style, and Description. The 'Web Tabs' section shows that no web tabs have been defined.

Custom Object Tabs

Action	Label	Tab Style	Description
Edit Del	Vehicle Customers	People	
Edit Del	Vehicle Dealers	Building	
Edit Del	Vehicle Orders	Box	
Edit Del	Vehicles	Car	
Edit Del	Vehicle Service Requests	Form	
Edit Del	Vehicle Test Drives	Gears	

Web Tabs

No Web Tabs have been defined

4. Creating the Lighting App

- A custom Lightning App named WhatNext Vision Motors was created.
- Included tabs: Vehicle, Vehicle Dealer, Vehicle Order, Vehicle Customer, Vehicle Test Drive, Vehicle Service
- Assigned to the System Administrator profile.

The screenshot shows the 'App Settings' page in the Lightning App Builder. The 'App Details & Branding' tab is selected. The 'App Details' section includes fields for 'App Name' (WhatNext Vision Motors), 'Developer Name' (WhatNext_Vision_Motors), and a 'Description' field. The 'App Branding' section shows an 'Image' upload button and a 'Primary Color Hex Value' (00070D2). Below these is an 'App Launcher Preview' showing a blue square with 'WV' and the text 'WhatNext Vision Motors'.

5. Creating Fields

Custom fields were created on each object to store specific business data.

For example, on the Vehicle__c object, a Picklist field named Vehicle_Model__c was created with values like *Sedan*, *SUV*, and *EV*.

Steps followed:

- Navigated to Setup → Object Manager → [Object Name] → Fields & Relationships → New.

The following tables represent the 'Fields & Relationships' setup for each object, as shown in the screenshots.

FIELD LABEL	FIELD NAME	DATA TYPE
Created By	CreatedBy	Lookup(User)
Last Modified By	LastModifiedBy	Lookup(User)
Owner	OwnerId	Lookup(UserGroup)
Price	Price__c	Currency(18, 0)
Status	Status__r	Picklist
Stock Quantity	Stock_Quantity__c	Number(18, 0)
Vehicle Dealer	Vehicle_Dealer__c	Lookup(Vehicle Dealer)
Vehicle Model	Vehicle_Model__c	Picklist
Vehicle Name	Name	Text(80)

FIELD LABEL	FIELD NAME	DATA TYPE
Address	Address__c	Text(80)
Created By	CreatedBy	Lookup(User)
Email	Email__c	Email
Last Modified By	LastModifiedBy	Lookup(User)
Owner	OwnerId	Lookup(UserGroup)
Phone	Phone__c	Phone
Preferred Vehicle Type	Preferred_Vehicle_Type__c	Picklist
Vehicle Customer Name	Name	Text(80)

FIELD LABEL	FIELD NAME	DATA TYPE
Created By	CreatedBy	Lookup(User)
Dealer Code	Dealer_Code__c	Auto Number
Dealer Location	Dealer_Location__c	Text(50)
Email	Email__c	Email
Last Modified By	LastModifiedBy	Lookup(User)
Owner	OwnerId	Lookup(UserGroup)
Phone	Phone__c	Phone
Vehicle Dealer Name	Name	Text(80)

FIELD LABEL	FIELD NAME	DATA TYPE
Assigned Dealer	Assigned_Dealer__c	Lookup(Vehicle Dealer)
Created By	CreatedBy	Lookup(User)
Last Modified By	LastModifiedBy	Lookup(User)
Order Date	Order_Date__c	Date
Owner	OwnerId	Lookup(UserGroup)
Status	Status__c	Picklist
Vehicle	Vehicle__c	Lookup(Vehicle)
Vehicle Customer	Vehicle_Customer__c	Lookup(Vehicle Customer)
Vehicle Order Name	Name	Text(80)

Vehicle Service Request			
Details	Fields & Relationships 8 Items, Sorted by Field Label		
Fields & Relationships	FIELD LABEL	FIELD NAME	DATA TYPE
Page Layouts	Created By	CreatedById	Lookup(User)
Lightning Record Pages	Issue Description	Issue_Description__c	Text(80)
Buttons, Links, and Actions	Last Modified By	LastModifiedById	Lookup(User)
Compact Layouts	Owner	OwnerId	Lookup(User/Group)
Field Sets	Service Date	Service_Date__c	Date
Object Limits	Status	Status__c	Picklist
Record Types	Vehicle	Vehicle__c	Lookup(Vehicle)
Related Lookup Filters	Vehicle Customer	Vehicle_Customer__c	Lookup(Vehicle Customer)
Search Layouts	Vehicle Service Request Name	Name	Text(80)
List View Button Layout			
Notification Rules			
Scoping Rules			

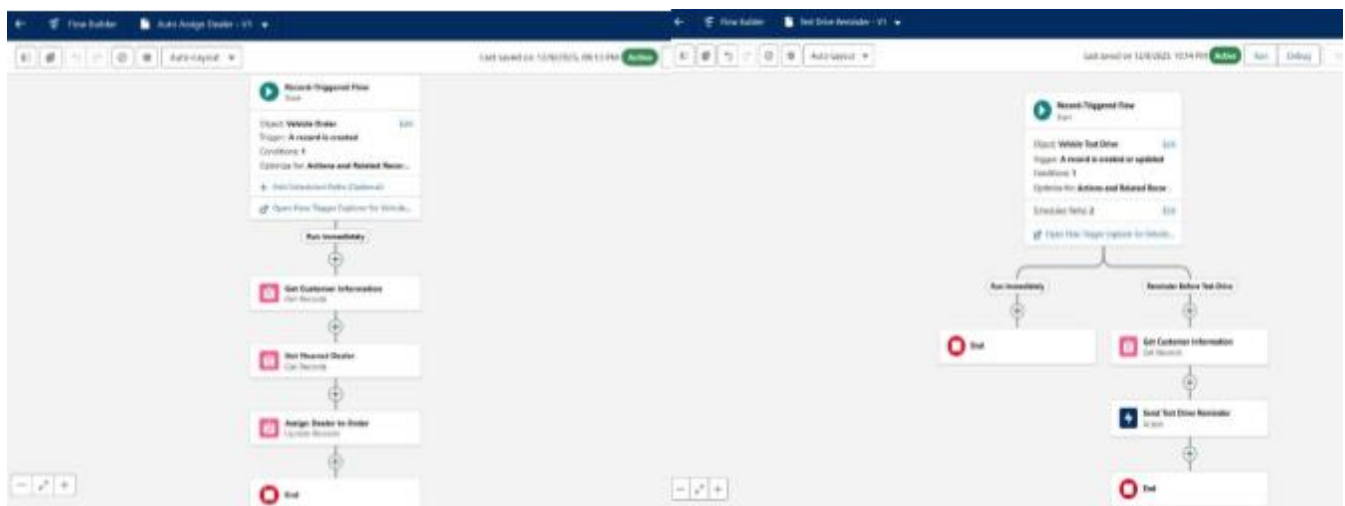
6. Creating Flows

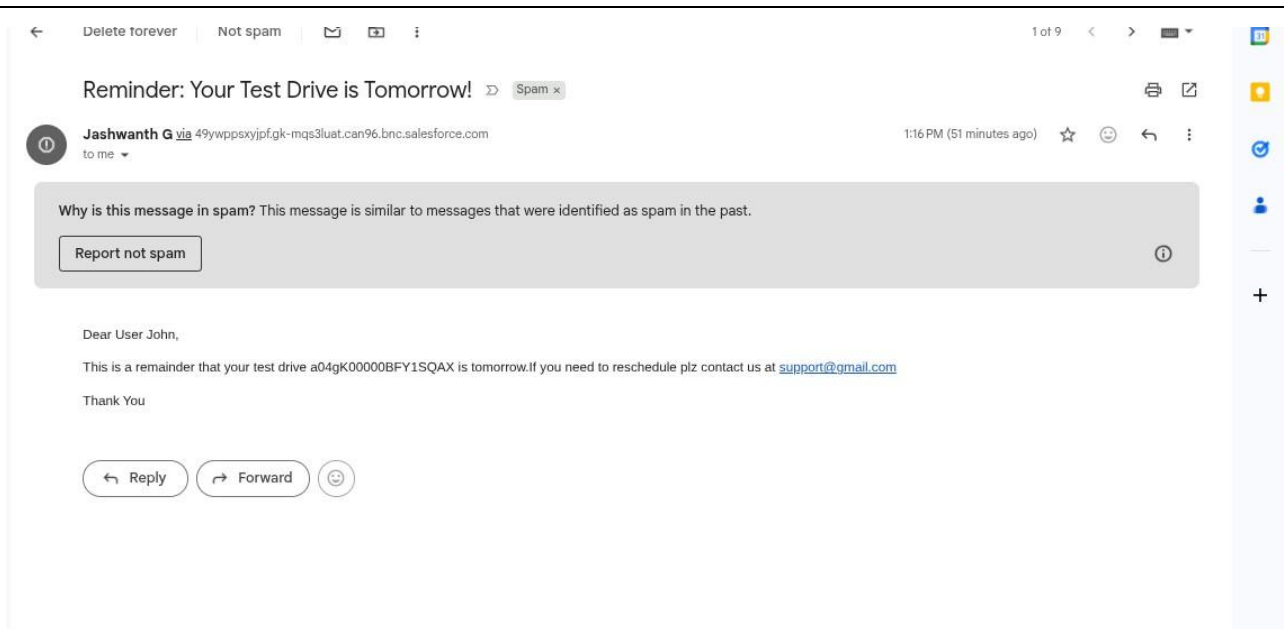
Record-triggered flows were created to automate key business processes without code. For example, a flow was built to **auto-assign the nearest dealer** when a vehicle order is created with status *Pending*.

Another flow sends **email reminders** to customers **one day before** their scheduled test drive.

Steps followed:

- Setup → Flows → New Flow → Record-Triggered Flow.





7. Create Apex and Trigger Batch Jobs:

Apex classes and triggers were created to enforce business logic during order processing.

- A **trigger** checks vehicle stock when an order is placed and updates stock on confirmation.
- A **batch class** regularly scans pending orders and updates them to *Confirmed* if stock becomes available.
- A **scheduled class** runs the batch job daily to automate order updates.

Steps followed:

- Developer Console → File → New → Apex Class.

```

orgfarm-55ce126fac-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage
File Edit Debug Test Workspace Help < >
VehicleOrderTriggerHandler.apxc VehicleOrderTrigger.apxt VehideOrderBatch.apxc VehicleOrderBatchScheduler.apxc
Code Coverage: None API Version: 65
1 public class VehicleOrderTriggerHandler {
2
3     public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> o
4         if (isBefore && (isInsert || isUpdate)) {
5             preventOrderIfOutOfStock(newOrders);
6         }
7
8         if (isAfter && (isInsert || isUpdate)) {
9             updateStockOnOrderPlacement(newOrders);
10        }
11    }
12
13    // ✖ Prevent placing an order if stock is zero
14    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
15        Set<Id> vehicleIds = new Set<Id>();
  
```

```
File Edit Debug Test Workspace Help < >
VehicleOrderTriggerHandler.apxc VehicleOrderTrigger.apxt VehicleOrderBatch.apxc VehicleOrderBatchScheduler.apxc
Code Coverage: None API Version: 65
1 trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert
2     VehicleOrderTriggerHandler.handleTrigger(trigger.new, trigger.oldMap, trigger.isBefore,
3 }
```

```
File Edit Debug Test Workspace Help < >
VehicleOrderTriggerHandler.apxc VehicleOrderTrigger.apxt VehicleOrderBatch.apxc VehicleOrderBatchScheduler.apxc
Code Coverage: None API Version: 65
1 global class VehicleOrderBatch implements Database.Batchable<sObject> {
2
3     global Database.QueryLocator start(Database.BatchableContext bc) {
4         return Database.getQueryLocator([
5             SELECT Id, Status__c, Vehicle__c FROM Vehicle_Order__c WHERE Status__c = 'Pending'
6         ]);
7     }
8
9     global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
10         Set<Id> vehicleIds = new Set<Id>();
11         for (Vehicle_Order__c order : orderList) {
12             if (order.Vehicle__c != null) {
13                 vehicleIds.add(order.Vehicle__c);
14             }
15         }
16
17         if (!vehicleIds.isEmpty()) {
```

```
orgfarm-55ce126fac-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage
File Edit Debug Test Workspace Help < >
VehicleOrderTriggerHandler.apxc VehicleOrderTrigger.apxt VehicleOrderBatch.apxc VehicleOrderBatchScheduler.apxc
Code Coverage: None API Version: 65
1 global class VehicleOrderBatchScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();
4         Database.executeBatch(batchJob, 50); // 50 = batch size
5     }
6 }
```

The screenshot shows a web application window titled "New Vehicle Order". The form contains several input fields: "Vehicle Order Number", "Vehicle Customer" (with value "John"), "Vehicle" (with value "Honda"), "Order Date" (with value "4/6/2026"), and "Status" (with value "Pending"). The "Status" field is highlighted in red, indicating an error. A red tooltip message appears over the "Status" field, stating: "We hit a snag. Review the errors on this page. This vehicle is out of stock. Order cannot be placed." The form also includes a "Save" button and a "Cancel" button. The background shows a blurred view of a browser window with a search bar and navigation icons.

PROJECT EXPLANATION WITH REAL-WORLD EXAMPLE

1. Customer

Registration Real-Life

Example:

A customer, **Elijah Mikaelson**, visits the automobile showroom or website.

In Salesforce:

- A new record is created in the **Vehicle_Customer_c** object with his details such as Name, Phone, Email, and Address.
- **Validation Rule:** Ensures the email is valid (e.g., must contain “@gmail.com”). If invalid, an error is shown.

Real-Life Example:

The store admin adds **cars like BMW X5, Toyota Fortuner, etc.**, with stock quantity (e.g., 5 BMWs and 3 Fortuners).

In Salesforce:

- Each vehicle is stored in the **Vehicle_c** object with details like Price, Model, and Stock Quantity.
- **Validation Rule:** Prevents adding a vehicle with negative stock values.

2. Vehicle & Inventory Management

Real-Life Example:

Elijah places an order for **1 BMW X5** (Price ₹50,00,000).

In Salesforce:

- A new record is created in the **Vehicle_Order_c** object.
- **Apex Trigger (VehicleOrderTrigger):**
 - Automatically calculates **Total Amount = Price × Quantity**.
 - Checks **stock availability**:
 - ☐ If stock > 0 → **Status_c = Confirmed**.
 - ☐ If stock = 0 → **Status_c = Pending**.

3. Order Placement

Real-Life Example:

When Elijah's order is confirmed, the BMW stock decreases by 1 (from 5 → 4).

In Salesforce:

- **Apex Trigger (VehicleOrderTriggerHandler):**
 - Updates **Vehicle__c.Stock_Quantity__c** by subtracting the quantity ordered.
 - Prevents stock from going below zero. **5.**

Pending

→ **Confirmed Orders (Stock Refill)**

Real-Life Example:

If BMW stock is 0 and Elijah's order was pending, when the admin refills **5 BMWs**, the system **automatically confirms pending orders** (if stock is available).

In Salesforce:

- **VehicleTrigger (After Update):**
 - Detects stock refill.
 - Calls `confirmPendingOrders()` in the **VehicleOrderTriggerHandler**.
 - Updates all pending orders to **Confirmed** until stock runs out. **6. Scheduled Batch Processing**

Real-Life Example:

At the end of each day, a **batch job** runs to update **any remaining pending orders** automatically.

In Salesforce:

- **VehicleOrderBatch & VehicleOrderBatchScheduler:**
 - Confirm pending orders if stock is available.
 - Sends admin a log of processed orders.

7. Email Notifications (Flow)

Real-Life Example:

When an order is **confirmed** or **pending**, Elijah gets an **email notification**.

In Salesforce:

- **Record-Triggered Flow:**
 - Sends an email alert:
 - “Your BMW X5 order is confirmed!” or
 - “Your order is pending due to low stock.”

8. Users and

Roles Real-Life

Example:

- **Niklaus Mikaelson** (Sales Manager) handles customer orders.
- **Kol Mikaelson** (Inventory Manager) manages vehicle stock

In Salesforce:

- Roles and Profiles:
 - **Sales Role:** Access to Vehicle_Order_c and Vehicle_Customer_c.
 - **Inventory Role:** Access to Vehicle_c and stock management.

9. Real-Life Benefit

This system ensures:

- **No manual stock checks** (triggers handle it automatically).
- **Automatic order status updates** (Confirmed or Pending).
- **Smooth communication with customers** via email alerts.
- **Dealer assignment** (optional via Flow) ensures orders are assigned to the nearest dealer.

SCREENSHOTS

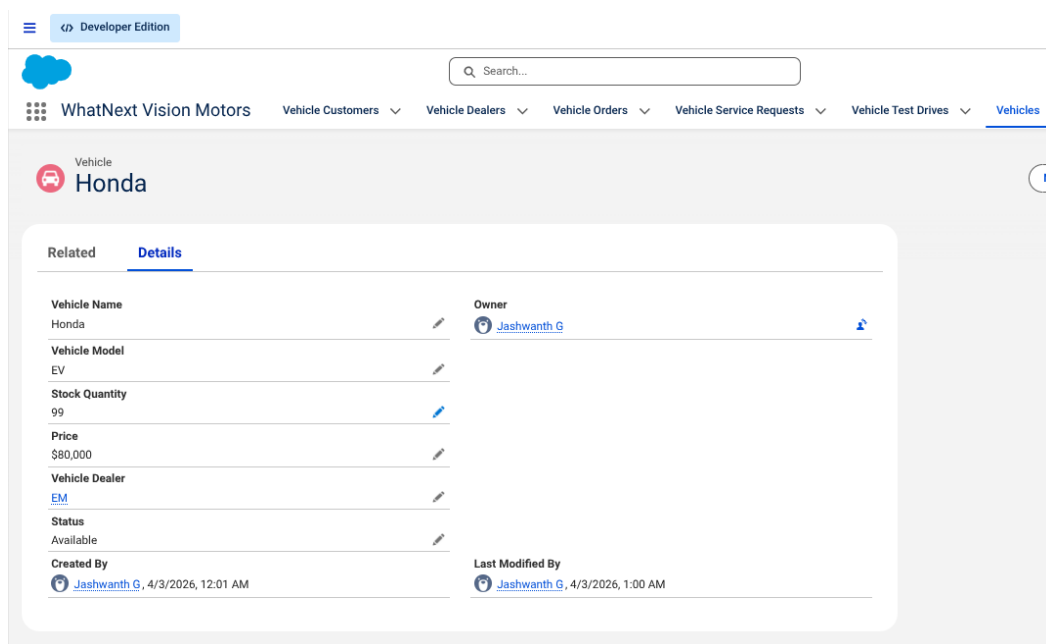


Fig: Vehicles in WhatNext Vision Motors

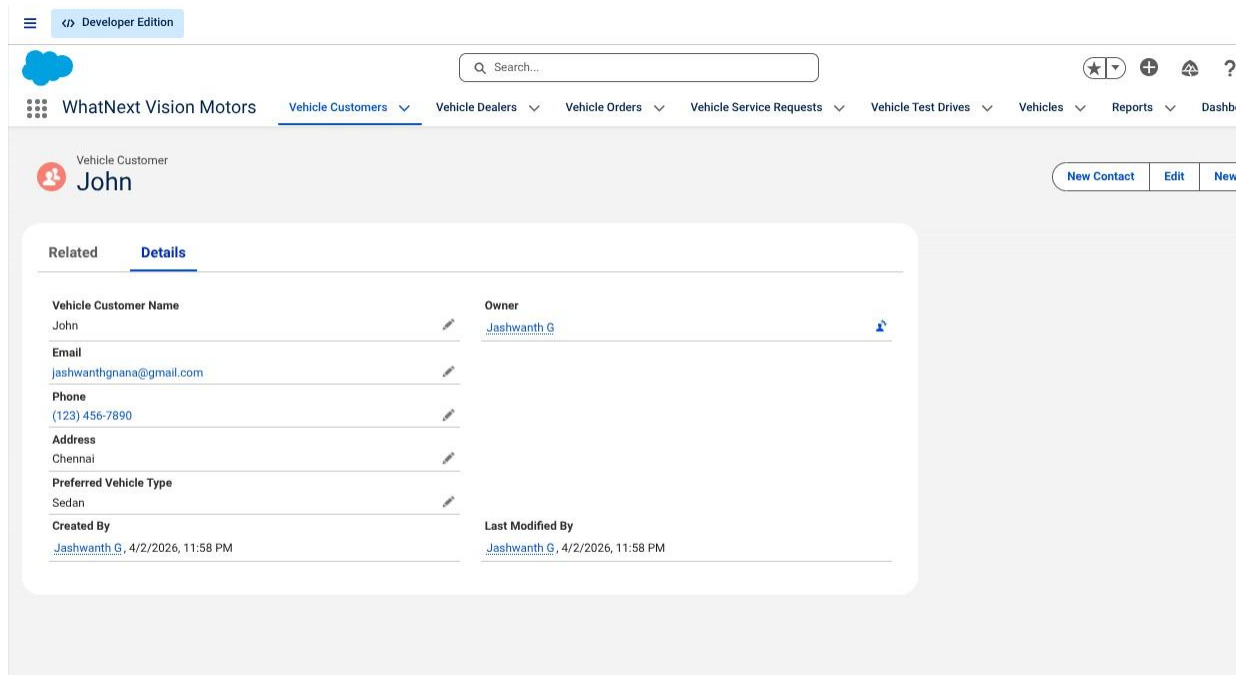


Fig: Customers in WhatNext Vision Motors

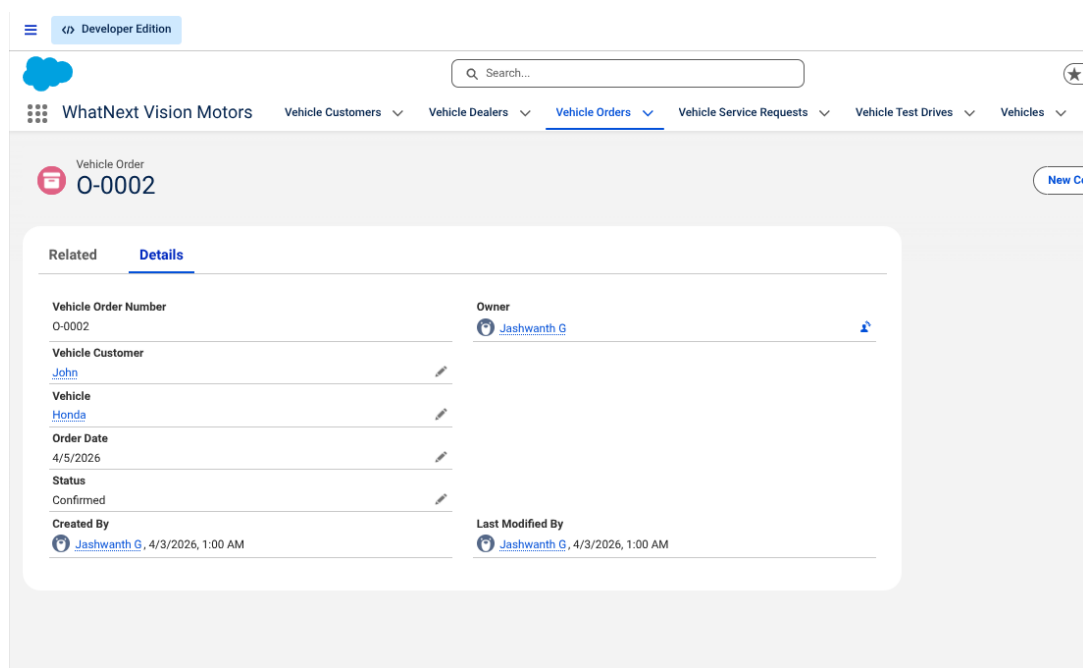


Fig: Vehicle Orders in WhatNext Vision Motors

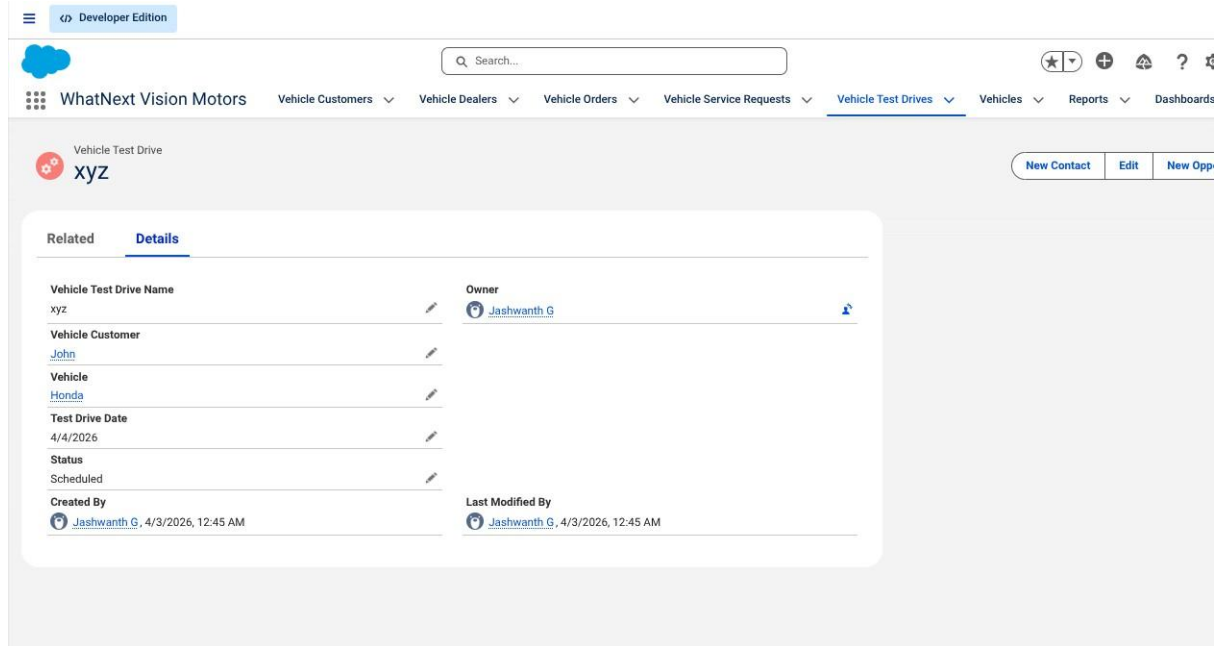


Fig: Test Drives in WhatNext Vision Motors

16. CONCLUSION

This project demonstrates the effectiveness of Salesforce CRM in automating real-world business processes. It significantly improves efficiency, enhances customer satisfaction, and provides a scalable solution for future growth.

17. REFERENCE:

Demo Video Link :

https://drive.google.com/file/d/1m3bVoZSsf3OUuGMDS8xbAlV0F4bi6YjS/view?usp=drive_link

Salesforce Link :

Team Lead - <https://www.salesforce.com/trailblazer/fkx2j9usaa6tbrvxl>

Member 1 - <https://www.salesforce.com/trailblazer/b97vxayrefnus2j38i>

Member 2 - <http://salesforce.com/trailblazer/dnseykjgaidry7whkp>

Member 3 - <https://www.salesforce.com/trailblazer/the1kyepg3r3girplw>

Github Repo Link : <https://github.com/Alex001-debug/NM-2026>